Introduction to Optimization

Cong Ma

University of Chicago, Winter 2026

An Optimization Problem

We write an optimization problem in the form

$$\min_{\boldsymbol{x} \in \mathcal{C}} \; f(\boldsymbol{x}),$$

where

- $\boldsymbol{x} \in \mathbb{R}^n$ is the *decision variable*,
- $f : \mathbb{R}^n \to \mathbb{R}$ is the *objective function*,
- $\mathcal{C} \subseteq \mathbb{R}^n$ is the *feasible set*.

Optimal solution: $\boldsymbol{x}^\star \in \mathcal{C}$ such that

$$f(\boldsymbol{x}^\star) \leq f(\boldsymbol{x}) \quad \forall \boldsymbol{x} \in \mathcal{C}.$$

Examples

- Least-squares and its regularized variants
- Matrix completion
- Empirical risk minimization and deep learning

These examples illustrate a recurring theme: model choice determines both statistical behavior and optimization difficulty.

Least-Squares Regression

Given data $(\boldsymbol{a}_i, b_i) \in \mathbb{R}^n \times \mathbb{R}$, $i = 1, \ldots, k$, solve

$$\min_{\boldsymbol{x} \in \mathbb{R}^n} \ \frac{1}{2k} \sum_{i=1}^{k} (\boldsymbol{a}_i^\top \boldsymbol{x} - b_i)^2$$

Equivalently,

$$\min_{\boldsymbol{x} \in \mathbb{R}^n} \ \frac{1}{2k} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2,$$

where

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{a}_1^\top \\ \vdots \\ \boldsymbol{a}_k^\top \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_k \end{bmatrix}.$$

Interpretation: fit a linear model by minimizing squared prediction error.

Least-Squares: Geometric View

- $\boldsymbol{A}\boldsymbol{x}$ lies in the column space of $\boldsymbol{A}$
- We are projecting $\boldsymbol{b}$ onto $\mathrm{col}(\boldsymbol{A})$

$$\boldsymbol{x}^\star = \arg\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2 \quad \Longleftrightarrow \quad \boldsymbol{A}\boldsymbol{x}^\star = \Pi_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b}$$

Key property: smooth, convex objective with a unique minimizer (when $\boldsymbol{A}$ has full column rank, equivalently $\boldsymbol{A}^\top\boldsymbol{A}$ is invertible).

Solving Least-Squares

Optimality condition:

$$\boldsymbol{A}^\top(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}) = 0 \quad \Longleftrightarrow \quad \boldsymbol{A}^\top\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^\top\boldsymbol{b}$$

Closed-form solution (when invertible):

$$\boldsymbol{x}^\star = (\boldsymbol{A}^\top\boldsymbol{A})^{-1}\boldsymbol{A}^\top\boldsymbol{b}$$

- Reliable and efficient numerical algorithms exist
- Computation depends on problem size and structure
- This is a *mature* optimization problem

Why Regularize?

Least squares can behave poorly when:

- features are highly correlated
- n is large relative to k
- data is noisy

Symptoms:

- large coefficients
- unstable predictions

Ridge Regression

$$\min_{\boldsymbol{x}\in\mathbb{R}^n} \ \frac{1}{2k}\|\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 + \lambda\|\boldsymbol{x}\|_2^2$$

- $\lambda > 0$ controls the strength of regularization
- Penalizes large coefficients

Equivalent view:

$$\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{x}\|_2^2 \leq t$$

Properties of Ridge Regression

- Objective is smooth and strongly convex
- Unique solution always exists
- Closed-form solution:

$$\boldsymbol{x}^\star = (\boldsymbol{A}^\top \boldsymbol{A} + 2k\lambda I)^{-1} \boldsymbol{A}^\top \boldsymbol{b}$$

- Improves numerical stability

Tradeoff: bias vs variance.

Lasso

$$\min_{\boldsymbol{x}\in\mathbb{R}^n} \ \frac{1}{2k}\|\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 + \lambda\|\boldsymbol{x}\|_1$$

- $\|\boldsymbol{x}\|_1 = \sum_{j=1}^n |x_j|$
- Encourages sparse solutions

Why Does Lasso Promote Sparsity?

- ℓ_1 penalty has *corners*
- Optimal solutions often occur at corners
- Corners correspond to zero coordinates

Consequence: automatic variable selection.

Optimization Perspective

- Objective is convex but **non-smooth**
- No closed-form solution
- Requires iterative algorithms

Key lesson: changing the regularizer changes both

- statistical behavior
- optimization difficulty

Motivation: The Netflix Challenge

- Netflix wants to recommend movies to users
- Users rate only a small fraction of movies
- Goal: *predict missing ratings accurately*
- This became the famous **Netflix Prize** problem

Key question: How do we infer a large number of missing entries from very few observations?

Data as a Matrix

- Let $\boldsymbol{M} \in \mathbb{R}^{n \times m}$
  - Rows = users
  - Columns = movies
  - $\boldsymbol{M}_{ij}$ = rating user i gives movie j
- We observe only entries in a set Ω

$$\Omega \subset \{1, \ldots, n\} \times \{1, \ldots, m\}$$

Task: Predict $\boldsymbol{M}_{ij}$ for $(i, j) \notin \Omega$.

Why Is This Hard?

- The matrix is huge (millions of users, thousands of movies)
- Most entries are missing (over 99%)
- Naively fitting all entries is impossible

Key insight: User preferences are *structured*

Low-Rank Structure

- Users can be described by a few latent factors
  - e.g. action vs romance, comedy vs drama
- Movies are also described by the same factors

$$\boldsymbol{M} \approx \boldsymbol{U}\boldsymbol{V}^\top$$

- $\boldsymbol{U} \in \mathbb{R}^{n \times r}$ (user factors)
- $\boldsymbol{V} \in \mathbb{R}^{m \times r}$ (movie factors)
- $r \ll \min(n, m)$

Conclusion: $\boldsymbol{M}$ is approximately **low rank**

Matrix Completion Problem

We only observe $\boldsymbol{M}_{ij}$ for $(i, j) \in \Omega$.

Ideal formulation:

$$\min_{\boldsymbol{X}} \ \mathrm{rank}(\boldsymbol{X}) \quad \text{s.t.} \quad \boldsymbol{X}_{ij} = \boldsymbol{M}_{ij}, \ (i, j) \in \Omega$$

- Matches observed ratings exactly
- Chooses the simplest (lowest-rank) explanation

Why Rank Minimization Is Hard

- Rank is:
  - Non-convex
  - Discontinuous
  - NP-hard to optimize
- Not suitable for large-scale problems

Optimization lesson: We often replace hard objectives with *tractable surrogates*

Convex Relaxation: Nuclear Norm

- Replace rank with the **nuclear norm**

$$\|\boldsymbol{X}\|_* = \sum_k \sigma_k(\boldsymbol{X})$$

- $\sigma_k(\boldsymbol{X})$ = singular values
- Convex envelope of rank on bounded sets

Convex formulation:

$$\min_{\boldsymbol{X}} \ \|\boldsymbol{X}\|_* \quad \text{s.t.} \quad \boldsymbol{X}_{ij} = \boldsymbol{M}_{ij}, \ (i,j) \in \Omega$$

Noisy Observations

In practice:

- Ratings are noisy
- Exact fitting is undesirable

Regularized formulation:

$$\min_{\boldsymbol{X}} \frac{1}{2} \sum_{(i,j)\in\Omega} (\boldsymbol{X}_{ij} - \boldsymbol{M}_{ij})^2 + \lambda \|\boldsymbol{X}\|_*$$

- Data fitting term
- Complexity control via nuclear norm

Alternative: Factorized Formulation

Instead of optimizing over $\boldsymbol{X}$ directly:

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{V}^\top$$

$$\min_{\boldsymbol{U},\boldsymbol{V}} \sum_{(i,j)\in\Omega} (\boldsymbol{u}_i^\top \boldsymbol{v}_j - \boldsymbol{M}_{ij})^2 + \lambda(\|\boldsymbol{U}\|_F^2 + \|\boldsymbol{V}\|_F^2)$$

- Non-convex
- Much more scalable
- Widely used in large-scale recommender systems

Clustering Example: Communities in Networks

- Data: a graph of interactions (friendships, emails, citations)
- Goal: partition nodes into **clusters/communities**
- Application: social networks, biology, recommender systems

Input: adjacency matrix $\boldsymbol{A} \in \{0,1\}^{n \times n}$, where $\boldsymbol{A}_{ij} = 1$ if there is an edge.

Stochastic Block Model (SBM)

- Each node has a hidden label $Z_i \in \{1, 2\}$ (community)
- Edges are generated independently given labels:

$$\mathbb{P}(\boldsymbol{A}_{ij} = 1 \mid Z_i = Z_j) = p, \qquad \mathbb{P}(\boldsymbol{A}_{ij} = 1 \mid Z_i \neq Z_j) = q$$

with $p > q$.

Inference goal: recover labels $(Z_1, \ldots, Z_n)$ from $\boldsymbol{A}$.

Optimization View: Maximum Likelihood

For balanced two-way clustering, let $\boldsymbol{x} \in \{\pm 1\}^n$ encode labels ($x_i = +1$ vs -1), and enforce balance $\mathbf{1}^\top \boldsymbol{x} = 0$.

A common surrogate objective (related to likelihood / cut):

$$\max_{\boldsymbol{x} \in \{\pm 1\}^n,\ \mathbf{1}^\top \boldsymbol{x} = 0} \quad \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}$$

- **Discrete, nonconvex** optimization
- Direct search is impossible when n is large

Relaxation Idea: From Discrete to Continuous

Relax the constraint $\boldsymbol{x} \in \{\pm 1\}^n$ to a sphere constraint:

$$\max_{\|\boldsymbol{x}\|_2^2 = n,\ \mathbf{1}^\top \boldsymbol{x} = 0} \quad \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}$$

This becomes an eigenvector problem:

- solution uses the **top eigenvector** of (centered) $\boldsymbol{A}$
- then **round** by $\mathrm{sign}(x_i)$

Optimization lesson: discrete $\rightarrow$ relaxation $\rightarrow$ rounding.

Empirical Risk Minimization (ERM)

Given data $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)$, ERM solves

$$\min_{\boldsymbol{\theta} \in \Theta} \quad \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i\big)$$

- $\boldsymbol{\theta}$ = model parameters
- $f_{\boldsymbol{\theta}}$ = prediction function
- ℓ = loss function

Unifying view: least squares, logistic regression, SVMs, neural networks.

ERM Includes Familiar Problems

- **Least squares:**

$$\ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y) = \tfrac{1}{2}(f_{\boldsymbol{\theta}}(\boldsymbol{x}) - y)^2, \quad f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x}$$

- **Logistic regression:**

$$\ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y) = \log(1 + \exp(-y f_{\boldsymbol{\theta}}(\boldsymbol{x})))$$

- **Regularization:**

$$\frac{1}{n}\sum_{i=1}^{n} \ell(\cdot) + \lambda \|\boldsymbol{\theta}\|_2^2$$

Same optimization template, different modeling choices.

Deep Learning = Large-Scale Optimization

In deep learning:

- $f_{\boldsymbol{\theta}}$ is a deep neural network
- $\boldsymbol{\theta}$ contains millions (or billions) of parameters
- The loss is typically nonconvex

$$\min_{\boldsymbol{\theta}} \quad \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i\big)$$

Key fact: training a neural network is an optimization problem.

Why Optimization Matters for Deep Learning

- Objective is **nonconvex**
- Data is **massive**
- Exact minimization is impossible

Yet simple methods like gradient descent work remarkably well.

Central questions:

- Why does optimization succeed?
- What structure are we exploiting?

A Recurring Pattern

- Start with a real-world problem
- Formulate an optimization problem
- Identify structure
- Choose algorithms accordingly

This pattern will repeat throughout the course.

Topics

- **Formulation:** convex optimization, nonconvex optimization
- **Algorithms:** gradient descent, Newton's method
- **Applications:** examples in data fitting, statistical estimation, geometric problems, etc
- **Duality Theory**